

Operaciones de aprendizaje automático

Actividad Individual | Deuda técnica, setup del stack y primer experimento con MLFlow

Iván Reyes Amezcua

Jorge David Alcázar Rochín- A01252855

20 de septiembre del 2026

1. Parte A. Deuda técnica en sistemas de aprendizaje automático

El paper de Sculley et al. (2015) parte de una idea que se me hizo incómoda pero real: el código propiamente de machine learning es una fracción muy pequeña de un sistema en producción, y todo lo que lo rodea (recolección de datos, configuración, servicio, monitoreo) es donde se acumula la deuda técnica. Le llaman deuda oculta porque no se ve en los indicadores normales de calidad de código y aparece después, cuando el sistema ya es caro de mantener. De los anti-patrones que describen, identifiqué tres que he visto de cerca en proyectos donde he participado como desarrollador.

1.1 Glue code (código pegamento)

Contexto. Trabajo en una aplicación móvil que digitaliza documentos impresos mediante OCR, a partir de fotos que toma el personal directamente en sitio. El reconocimiento lo hace una librería de propósito general que para nosotros es prácticamente una caja negra. Alrededor de esa llamada tenemos código de soporte para preparar la imagen (recorte, rotación, contraste), enviarla, y después limpiar el texto que regresa con expresiones regulares, catálogos de nombres y reglas distintas por tipo de documento. La parte de ML son unas cuantas líneas y el código que la rodea son cientos.

Consecuencias. El sistema queda amarrado a las particularidades de esa librería. Cambiar de proveedor de OCR implicaría rescribir casi toda la capa de limpieza, así que en la práctica nadie lo propone. Además ese código pegamento es el que mas se rompe cuando cambia algo mínimo, y no tiene pruebas porque nunca se pensó como un componente formal.

1.2 Pipeline jungles (junglas de tuberías)

Contexto. En otro proyecto, de levantamiento de encuestas, los datos llegan desde la app móvil, se sincronizan al backend y de ahí salen los reportes para el cliente. Ese camino no se diseño de una sola vez: fue creciendo con scripts sueltos para cruzar catálogos, corregir capturas mal hechas y exportar a Excel. Es justo lo que el paper describe como un pipeline que evoluciona de forma orgánica.

Consecuencias. Solo una o dos personas saben en qué orden hay que ejecutar las cosas. Si algo falla a la mitad hay que repetir el proceso completo, y reproducir un

reporte que se entregó hace meses es casi imposible porque los datos y los scripts ya cambiaron. Esto se conecta con lo que el paper llama deuda de reproducibilidad.

1.3 Umbrales fijos en sistemas dinámicos y deuda de configuración

Contexto. En el mismo proyecto de OCR existe un umbral de confianza que decide si un campo leído se acepta automáticamente o se manda a validación manual. Ese valor se definió a ojo cuando se hicieron las primeras pruebas y quedó repartido entre una variable de entorno y un par de constantes en el código. Nadie documentó por qué es ese número y no otro.

Consecuencias. Cuando cambian las condiciones (un formato de documento distinto, celulares con otra cámara, fotos con menos luz) el umbral deja de ser el adecuado y el sistema empieza a aceptar lecturas malas o a mandar demasiado trabajo a revisión manual, sin que nadie se entere hasta que el usuario se queja. Tener la configuración regada en varios lugares hace que ajustarlo sea riesgoso, porque no hay forma sencilla de saber qué versión estaba corriendo con qué umbral.

1.4 Qué problema concreto resolvería MLOps

El problema concreto en estos proyectos es que no podemos responder con certeza por qué una versión se comporta peor que la anterior. Si la precisión del OCR baja, hoy la discusión es de opiniones: unos dicen que fue el cambio de librería y otros que las fotos vienen peor. No hay un registro de qué datos, qué configuración y qué versión produjeron cada resultado, y por eso cada mejora se vuelve una apuesta.

MLOps atacaría justamente eso. Registrar cada corrida con sus parámetros, sus métricas y sus artefactos, que es lo que se practica en la Parte C de esta actividad con MLflow Tracking, convierte esa discusión en algo que se puede consultar y comparar. Versionar los datos y los modelos junto con el código ataca la deuda de reproducibilidad de la jungla de tuberías. Declarar el pipeline en una herramienta, en lugar de scripts sueltos, reduce el código pegamento y la dependencia de que una sola persona sepa el orden. Y monitorear las métricas en producción permitiría detectar que el umbral ya no sirve antes de que lo note el cliente, y no después.

Lo que me queda de la lectura es que la deuda técnica de ML no se paga escribiendo mejor código, sino cambiando la forma de trabajar: tratar los datos y la configuración como parte del sistema y no como algo que vive aparte.

2. Parte B. Setup del stack del curso

Se instalaron y verificaron los componentes del stack del curso sobre un entorno Unix (macOS): Python, Docker, Git y MLflow. La Figura 1 muestra la salida de la terminal con la versión de cada componente instalado.

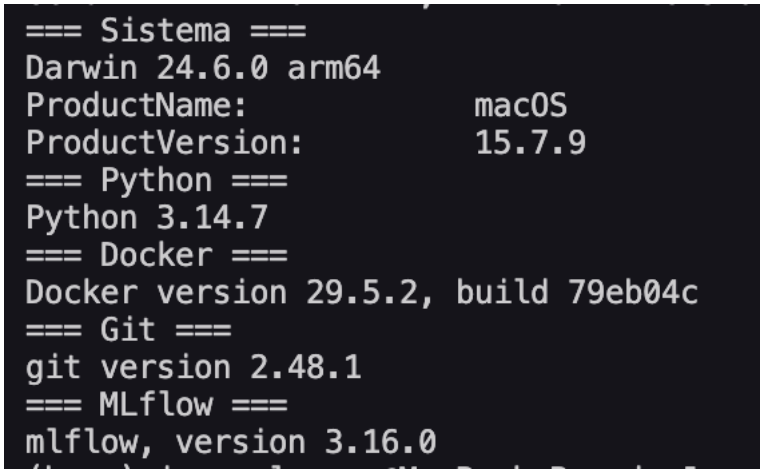


Figura 1. Versiones de los componentes del stack instalados en el entorno local.

Después se ejecutó el comando mlflow ui en la terminal y se confirmó que la interfaz gráfica levanta correctamente en http://localhost:5000. La Figura 2 muestra la interfaz de MLflow recién iniciada, todavía sin experimentos registrados.

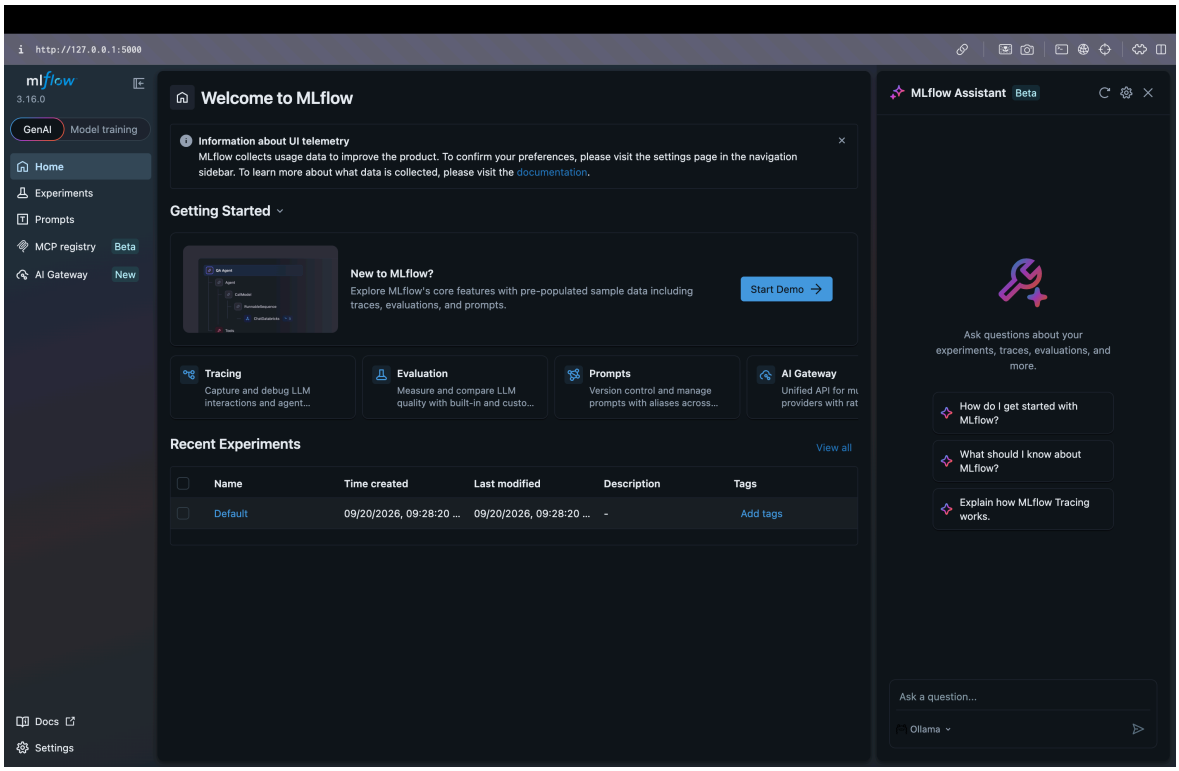


Figura 2. MLflow UI corriendo en localhost:5000 sin experimentos registrados.

3. Parte C. Hello World en MLflow

Se creó el script hello_mlflow.py, que registra un experimento llamado hello_mlflow con un run de prueba. El script registra tres parámetros con log_param (modelo, learning_rate y epochs), dos métricas por época con log_metric (loss y accuracy) más sus valores finales, y un artefacto de texto con log_artifact. Como backend de tracking se usó una base de datos SQLite local.

La salida del script en la terminal fue la siguiente:

```
epoca 0: loss=0.7128  accuracy=0.2872
epoca 1: loss=0.4995  accuracy=0.5005
epoca 2: loss=0.3551  accuracy=0.6449
epoca 3: loss=0.2530  accuracy=0.7470
epoca 4: loss=0.1919  accuracy=0.8081

Run registrado correctamente.
```

Al recargar la interfaz gráfica, el experimento aparece registrado con sus corridas, como se ve en la Figura 3.

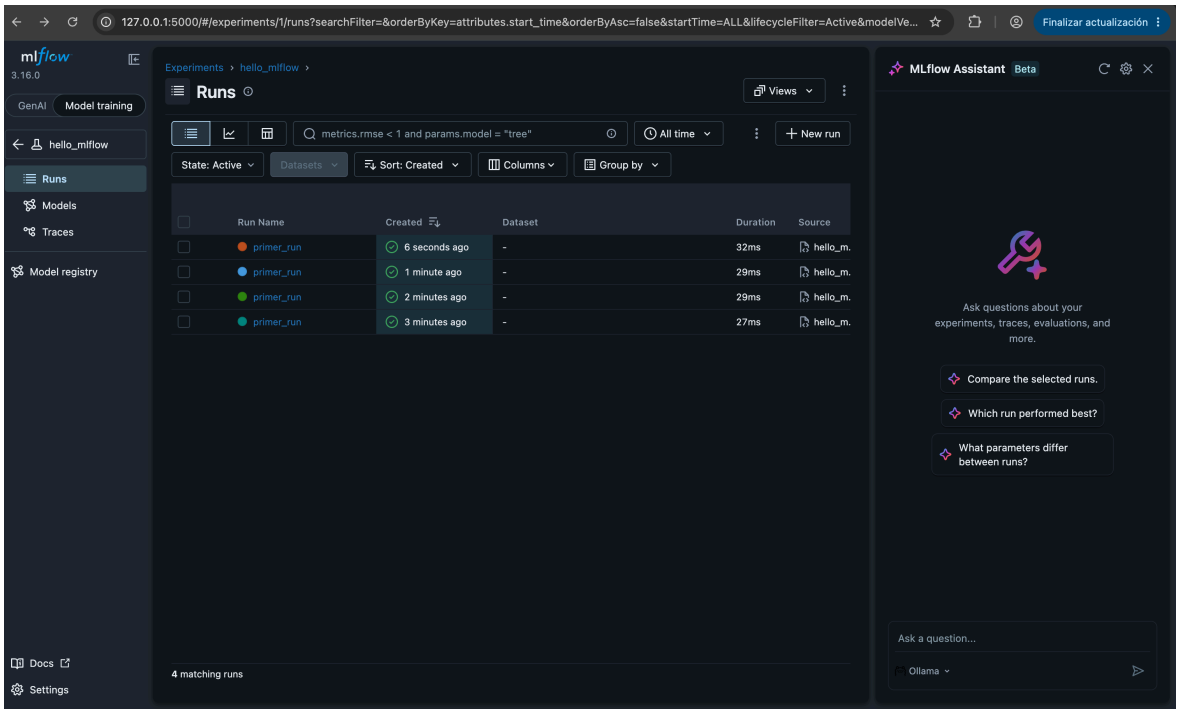


Figura 3. Experimento hello_mlflow con sus corridas registradas en la MLflow UI.

Entrando al detalle de la corrida se pueden ver los parámetros y las métricas que se registraron (Figura 4), así como el artefacto resultados.txt que generó el script (Figura 5).

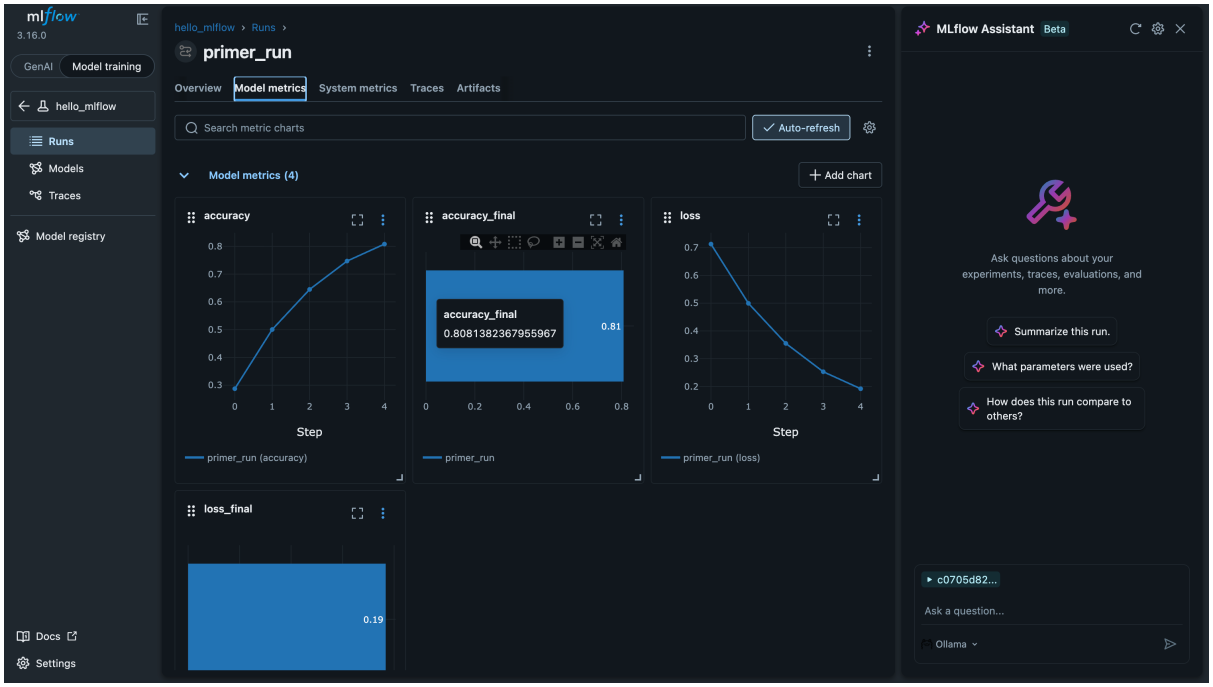


Figura 4. Métricas de la corrida primer_run registradas en la MLflow UI.

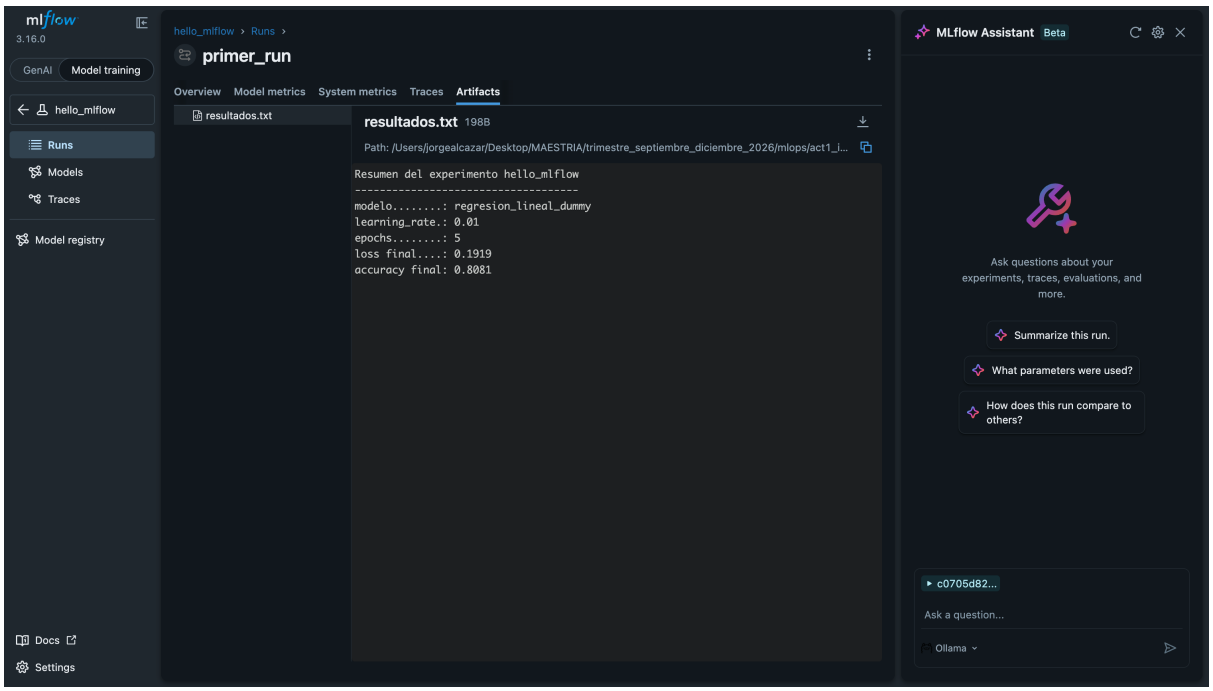


Figura 5. Pestaña Artifacts con el archivo resultados.txt registrado por el script.

4. Apéndice. Código del script hello_mlflow.py

A continuación se incluye el código completo del script utilizado en la Parte C.

```
import random
from pathlib import Path
import mlflow


mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("hello_mlflow")

with mlflow.start_run(run_name="primer_run"):


    learning_rate = 0.01
    epochs = 5

    mlflow.log_param("modelo", "regresion_lineal_dummy")
    mlflow.log_param("learning_rate", learning_rate)
    mlflow.log_param("epochs", epochs)



    random.seed(42)
    loss = 1.0
    accuracy = 0.0

    for epoch in range(epochs):
        loss = loss * 0.7 + random.uniform(0.0, 0.02)
        accuracy = 1.0 - loss
        mlflow.log_metric("loss", loss, step=epoch)
        mlflow.log_metric("accuracy", accuracy, step=epoch)
        print(f"epoca {epoch}: loss={loss:.4f}  accuracy={accuracy:.4f}")

    mlflow.log_metric("loss_final", loss)
    mlflow.log_metric("accuracy_final", accuracy)



    reporte = Path("resultados.txt")
    reporte.write_text(
        "Resumen del experimento hello_mlflow\n"
        "-----------------------------------\n"
        "modelo........: regresion_lineal_dummy\n"
        f"learning_rate.: {learning_rate}\n"
        f"epochs........: {epochs}\n"
        f"loss final....: {loss:.4f}\n"
        f"accuracy final: {accuracy:.4f}\n",
        encoding="utf-8",
    )
    mlflow.log_artifact(str(reporte))

    run_id = mlflow.active_run().info.run_id
    print("\nRun registrado correctamente.")
```

5. Referencias

MLflow. (2024). MLflow Tracking. MLflow Documentation. https://mlflow.org/docs/latest/tracking.html

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J., & Dennison, D. (2015). Hidden technical debt in machine learning systems. En Advances in Neural Information Processing Systems (Vol. 28). Curran Associates, Inc.